

Certificazione Slope - Preparazione al colloquio

SANTADDEO Hotel Accelerator - 4BID S.r.l. | 22 luglio 2026

Contesto: Slope certifica la nostra integrazione con la Partner API. Parte 1: domande che il loro team tecnico potrebbe farci, con le risposte basate sull'implementazione reale del connettore (verificata end-to-end in sandbox staging). Parte 2: domande che conviene porre noi a Slope.

Parte 1 - Le loro possibili domande, le nostre risposte

1. Autenticazione e configurazione

D: Come gestite l'autenticazione verso la Partner API?

R: Bearer token nell'header Authorization: bearer <token>. Il token e' scoped per struttura: ogni hotel in Santaddeo ha la propria API key, nessun establishment/structure ID separato. Supportiamo produzione (api.slope.it) e staging (api.staging.slope.it) con endpoint configurabile per hotel.

D: Come conservate i token?

R: In database (tabella integrazioni PMS per-hotel), mai esposti lato client; tutte le chiamate Slope partono esclusivamente dal backend. Mai usiamo il token sandbox su strutture reali e viceversa.

2. Sincronizzazione prenotazioni

D: Quale strategia di sync usate?

R: La 'Strategia 1' della vostra documentazione: delta sync incrementale su GET /v1/lodging-reservations con filter=lastUpdateDate:gt:< cursore>. Cursore persistente per hotel; primo giro full sync, poi solo delta. Verificato: al secondo giro con cursore aggiornato riceviamo 0 record se non ci sono modifiche.

D: Come gestite le cancellazioni?

R: Su due livelli: cancellazioni soft via campo isCanceled della reservation, e hard delete interrogando GET /v1/deleted-resources?resourceType=lodgingReservation a ogni ciclo. Entrambe marcano la prenotazione come cancellata nel nostro datastore (flag, non eliminazione).

D: Come gestite la paginazione?

R: Seguiamo pagination.nextPage nella risposta { data: [...], pagination: {...} } fino a esaurimento, per ogni finestra di sync.

D: Quali expand usate?

R: expand=lodgingType,pricesByDate,primaryGuest,ratePlansByDateRange - come scalari separati da virgola (lo stile array expand[] restituisce 400).

D: Con che frequenza sincronizzate?

R: Cron schedulato (oraria di default, configurabile per hotel) piu' sync manuale on-demand dal pannello. Grazie al delta sul cursore, il carico per giro e' minimo.

3. Rate limiting e resilienza

D: Come rispettate il rate limit di 30 req/min?

R: Client con retry e backoff esponenziale e gestione esplicita del 429 (attende e ritenta). Il delta sync riduce drasticamente le chiamate: a regime un giro di sync sono poche richieste. Errori e timeout incapsulati in una classe di errore dedicata con logging; un fallimento di sync non corrompe il cursore (avanza solo a successo).

D: Cosa succede se una sync fallisce a meta'?

R: Il cursore non viene avanzato: il giro successivo riprende dagli stessi dati. Le scritture raw sono upsert idempotenti (chiave univoca hotel + reservation id), quindi nessun duplicato.

4. Push prezzi e disponibilita'

D: Come inviate i prezzi?

R: POST /v1/lodging-types/{id}/rates-and-availability-updates. Gestiamo la semantica asincrona della risposta (HTTP 202 = accettato). I valori rates[].rate sono serializzati come stringa Money a 2 decimali (es. "150.00"), come richiesto dalla vostra validazione.

D: Pushate sulle tariffe derivate?

R: No. Leggiamo isDerived da /v1/rate-plans e pushiamo solo sui piani non derivati: le derivate si calcolano nel vostro sistema. Stessa regola che applichiamo con gli altri PMS.

5. Mappatura dati

D: Come mappate tipologie e tariffe?

R: GET /v1/lodging-types e /v1/rate-plans con mapping esplicito per-hotel verso le nostre entita' (colonne dedicate slope_lodging_type_id, slope_rate_plan_id). Gestiamo i campi name multilingua ({{locale, value}}) con fallback it -> en -> primo disponibile.

D: Quali dati della prenotazione consumate?

R: Date di check-in/out, prezzi per notte (pricesByDate), numero camere, adulti/bambini, ospite principale, canale di vendita (saleSource mappato sui nostri codici canonici: DIRECT -> Diretto, ONLINE_SALES_CHANNEL -> OTA, ecc.), stato cancellazione.

6. Test e qualita'

D: Avete testato in sandbox?

R: Si', test end-to-end completo su staging (backoffice.staging.slope.it): 12 prenotazioni sincronizzate con prezzi/notti/ospiti/canale corretti, delta sync con cursore verificato, deleted-resources verificato, push prezzi accettato con 202. Abbiamo inoltre un pannello di health-check interno che testa periodicamente ogni endpoint del connettore.

D: Casi limite noti?

R: Si', gestiti esplicitamente: filtro lastUpdateDate senza millisecondi (con ms la API risponde 400 - tronchiamo il cursore in scrittura e lettura), stile scalare di filter/expand, validazione stretta degli UUID su deleted-resources.

7. Domande di processo

Volumi previsti: poche strutture inizialmente (es. Hotel Verdi), crescita graduale; carico API minimo grazie al delta sync.

Ambienti: sviluppo/test solo su staging con credenziali sandbox; produzione solo con token di produzione della struttura.

Onboarding nuova struttura: inserimento token -> discovery tipologie/tariffe -> mapping -> sync iniziale -> attivazione cron. Nessuna azione lato Slope oltre all'emissione del token.

Supporto/contatti: riferimento tecnico 4BID con accesso sandbox gia' attivo (connect+4bid@slope.it).

Nota importante

Non consumiamo un endpoint di disponibilita' (la Partner API non lo espone): la nostra occupancy e' derivata internamente dalle prenotazioni. In scrittura usiamo l'endpoint rates-and-availability solo per i prezzi.

Parte 2 - Domande da porre noi a Slope

Priorita' alta (limiti reali che abbiamo oggi)

1. Esponete (o avete in roadmap) un endpoint di disponibilita'/inventario?

Perche': oggi la Partner API non lo espone e deriviamo l'occupancy dalle prenotazioni. Così non possiamo distinguere le camere fuori servizio/in manutenzione da quelle libere. Chiedere se esiste un feed di disponibilita', blocchi, stop-sell o room out-of-order, anche solo in roadmap.

2. Avete webhook/notifiche push per le modifiche alle prenotazioni?

Perche': oggi facciamo polling con delta sync (cursore su lastUpdateDate). Funziona, ma un webhook su create/update/cancel ridurrebbe la latenza da 'fino a un'ora' a quasi zero e il consumo di API quasi a niente. Se esistono: payload, retry policy e firma di sicurezza.

3. Il push prezzi e' asincrono (202): come verifichiamo l'esito?

Perche': riceviamo 'accettato' ma non sappiamo se l'applicazione e' poi andata a buon fine. Esiste un endpoint per interrogare lo stato dell'update, o un webhook di conferma/errore? Sui nostri altri PMS abbiamo un 'Price Guard' che confronta pushato vs applicato e vorremmo la stessa affidabilita' su Slope.

4. Possiamo scrivere anche restrizioni oltre ai prezzi?

Perche': l'endpoint si chiama rates-and-availability-updates ma lo usiamo solo per i rate. Supporta minimum stay, closed-to-arrival/departure, stop-sell, apertura/chiusura vendite? Se si, si aprono funzionalita' di revenue management aggiuntive.

Priorita' media (operativita' e scala)

5. Il rate limit di 30 req/min e' per token/struttura o per partner (IP)?

Perche': con una struttura e' irrilevante, ma con 20 hotel Slope in parallelo cambia il dimensionamento dei nostri cron. Chiedere anche se e' negoziabile per partner certificati.

6. Qual e' il processo di go-live per una nuova struttura comune?

Perche': serve il flusso concreto: chi chiede il token (noi o l'hotel?), tempi di emissione, esiste un portale partner o si passa da connect@slope? Il token ha scadenza/rotazione?

7. La retention dei deleted-resources e' limitata nel tempo?

Perche': se un nostro sync resta fermo (manutenzione, incidente) per N giorni, gli hard-delete piu' vecchi di N restano recuperabili o li perdiamo? Determina il nostro requisito di massimo downtime senza riconciliazione completa.

8. Ci sono campi per identificare tariffe non pushabili oltre a isDerived?

Perche': su altri PMS esistono tariffe 'chiuse', pacchetti o rate gestite dal channel manager su cui non bisogna scrivere. Se Slope ha concetti simili (rate plan read-only, pacchetti), meglio saperlo prima di scrivere su una struttura reale.

Priorita' bassa (nice-to-have)

9. Ambiente staging: i dati sandbox restano disponibili a lungo termine?

Perche': ci serve per i test di regressione continui del nostro pannello di health-check.

10. Versionamento API: come comunicare breaking change e deprecazioni?

Perche': mailing list partner, changelog, header di versione?

11. Esiste un endpoint per l'anagrafica struttura (nome, indirizzo, camere totali)?

Perche': da usare in onboarding invece dell'inserimento manuale.

12. La lista dei valori possibili di saleSource e' stabile e documentata?

Perche': ne abbiamo mappati alcuni sui nostri codici canonici e vogliamo essere certi di non perderne di nuovi.

Sintesi strategica

Le domande 1 e 3 sono le piu' importanti: senza disponibilita' reale il nostro motore lavora su occupancy derivata (con il limite sulle camere fuori servizio), e senza conferma dell'esito del push non possiamo garantire l'integrita' prezzi come facciamo sugli altri PMS.